

Sql training

1 intro

sql(structured Query Language) is used to manage ,query and manipulate relational Database in enterprise environments.

Core categories:DDL,DML,TCL,DCL

2,create database and tables

create database db

use db

3.insert values

```
create Table employees(  
emp_id INT PRIMARY KEY,  
name VARCHAR(50),  
department VARCHAR(50),  
salary DECIMAL(10,2),  
hire_date DATE);
```

4.sql statement fundamental

```
select CURRENT_TIMESTAMP;  
select UPPER(name) from employees  
select LEN(name) from employees  
select count(*) from employees  
select avg(salary) from employees  
select sum(salary) from employees
```

5.joins and sql

```
SELECT emp.name  
FROM employees as emp  
LEFT JOIN dept  
ON emp.department = dept.dept_name;
```

```
SELECT emp.name  
FROM employees as emp  
right JOIN dept  
ON emp.department = dept.dept_name;
```

```
SELECT emp.name  
FROM employees as emp  
inner JOIN dept  
ON emp.department = dept.dept_name;
```

6.groupby statement and mathematical function

```
select department,count(*)  
from employees  
group by department;
```

```
select ROUND(AVG(salary),2) FROM employees  
select MAX(salary),min(salary) from employees
```

7.view

```
create view high_salary AS  
select name,salary from employees  
where salary>10000;  
select * from high_salary
```

8.subqueires

```
select name,salary from employees  
where salary>(select AVG(salary) from employees)
```

```
select name from employees  
where department in(  
select dept_name from dept where dept_name='cs')
```

Hands on practice

1.Retrieve all data

Specific with condition

Use where,orderby,limit

```
select * from employees
```

```
select hire_date from employees  
where hire_date>'05-01-2021'
```

```
select * from employees  
order by hire_date
```

```
select * from employees  
order by hire_date desc
```

```
select top 1 * from employees  
order by hire_date
```

2.differnce between delete,truncateand ,drop

```
DELETE FROM employees  
WHERE emp_id==4;
```

drop table dept

truncate table dept

What is normalization?

2. creating databases and table

with ,constraints:primary key,not null,unique

```
create Table student(  
std_id INT PRIMARY KEY,  
name VARCHAR(50) not null,  
department VARCHAR(50),  
salary DECIMAL(10,2),  
ph VARCHAR(50) unique);
```

```
insert into student values(1,'sam','cs',10000,99999999)  
insert into student values(2,'sann','cs',10000,99999998)  
insert into student values(3,'sammy','ec',10000,99999997)  
insert into student values(4,'amy','ec',10000,99999996)  
insert into student values(5,'anny','cs',10000,99999995)  
insert into student values(6,'manny','cs',10000,99999994)  
insert into student values(7,'aby','eee',10000,99999993)  
insert into student values(8,'babu','eee',10000,99999992)  
insert into student values(9,'tom','eee',10000,99999991)  
select * from student
```

3.sql

timestamp:CURRENT_TIMESTAMP

String functions UPPER(),LOWER(),CONCAT(),LEN()

Aggregate function:COUNT(),SUM(),AVG(),MIN(),MAX()

```
select CURRENT_TIMESTAMP;
```

```
select UPPER(name) from student  
select LOWER(name) from student  
select CONCAT(name,' ',department) as dp from student  
select LEN(name) from student  
select count(*) from student  
select avg(salary) from student  
select sum(salary) from student  
select min(salary) from student  
select max(salary) from student
```

4.joins

Inner join ,left join, right join,full join

Difference between left and inner join?

Inner join -Returns only rows where there is a match in both tables.

Left join -Returns all rows from the left table, and matching rows from the right table.

What is a self join ?

A self join in SQL is when a table is joined with itself. It's useful when you want to compare rows within the same table or find relationships among data stored in a single table.

```
SELECT student.name,student.department_id,student.ph  
FROM student  
LEFT JOIN dep  
ON student.department_id = dep.dep_id;
```

```
SELECT student.name,student.department_id,student.ph  
FROM student  
right JOIN dep  
ON student.department_id = dep.dep_id;
```

```
SELECT student.name,student.department_id,student.ph  
FROM student  
full JOIN dep  
ON student.department_id = dep.dep_id;
```

5.group by

```
select department_id,count(*) from student  
group by department_id
```

1.count employees count per department

```
select department_id,count(*) from student  
group by department_id  
having department_id >1
```

2.find department with salary>5000

```
select department_id,avg(salary) from student  
group by department_id  
having avg(salary)>5000;
```

Group by with having

```
round(),ceiling(),floor()
```

```
select ceiling(avg(salary)) from student  
select floor(avg(salary)) from student  
select round(avg(salary),4) from student
```

6.view

```
create view high as  
select * from student where salary>5000
```

```
select * from high;
```

7.subquieres-independent

Subquieres inside where,from,and select clauses

Use IN,EXISTS,and NOT EXISTS

```
select * from student  
where std_id in(1,2)
```